WEEK 08

Advanced Interrupt Software Design

Writing the handler, and the three ways a shared variable gets corrupted between two instructions

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 4, printed pages 122–132.
Lab manual: Lab 03 Section 3.7.4 (writing the handler), Lab 04 Section 4.4 (timer ISR).

1 What we are actually after this week

Last week we built the hardware chain that gets an ISR called. This week we write the ISR, and then we deal with the consequence we deliberately postponed.

The consequence is this. An interrupt can fire **between any two machine instructions**. Your task code and your handler both touch the same global variables. So what happens when the handler lands in the middle of the task's arithmetic?

The answer is: something wrong, in three distinct ways, and the three need three different fixes. That is the spine of this week.

1. **Partitioning.** How much work belongs in the ISR versus the main-line code. Three options, and why the middle one usually wins.
2. **Writing the handler in C.** The rules, the signature, and the flag-clearing discipline.
3. **Corruption type 1: the stale register.** The compiler caches a value the ISR then changes. Fixed by `volatile`.
4. **Corruption type 2: the read-modify-write race.** Two increments produce one. Fixed by a critical section. `volatile` does *not* help.
5. **Corruption type 3: the partial update.** A 64-bit variable half old and half new. Fixed by a critical section, and by not using 64-bit shared variables.
6. What a **critical section** actually is, and the cost of one.

Items 4 and 5 are the ones students get wrong on exams, because almost everybody believes `volatile` makes shared data safe. It does not. Getting that distinction crisp is the single most valuable thing in this week's notes.

2 Deep dive 1: Partitioning, or how much goes in the ISR

Before writing code, a design decision. Recall the flasher from Week 05: pressing a switch must identify which switch changed, determine its new value, update `g_w_delay`, `g_RGB_delay`, and `g_flash_LED`, and eventually light the LEDs.

How much of that belongs in the handler?

The partitioning rule

> An ISR should perform only **quick, urgent** work related to the interrupt. Other work should be deferred to main-line code where feasible. This keeps every ISR short, which avoids delaying other ISRs, and keeps the code debuggable.

The book lays out three options and they are worth walking, because the trade-off recurs constantly.

2.1 Option 1: short ISR

The handler does the minimum: record which switch changed and whether it was pressed or released. Everything else happens in a task.

Sounds clean, and it creates awkward questions. Which task updates the delay variables? If `Task_RGB` and `Task_Flash` do it, they now contain switch-handling logic, which wrecks the modularity we built in Week 05. If a new task does it, you pay the overhead of another task in the scheduler.

Worse, the handoff needs protocol. How does the task know not to reuse stale switch data next time round? Does the ISR set a flag saying "run once"? Does the task erase the data when it is done? And what if the ISR runs *twice* before the task gets a chance: should the task process the first event, the last, or all of them?

Beware the trap

That last question has no universally right answer, it depends on the data.

For a **switch position**, only the latest value matters. Losing intermediate presses is acceptable, arguably correct.

For **serial communication**, every byte matters. Losing one corrupts the message. So the ISR and the task must agree on somewhere to *accumulate* data, which is what a **queue** is for, and which is why Week 14 spends real time on queue implementation before it gets to UART.

Ask yourself which kind of data you have before choosing a partitioning. "Latest value wins" and "every value counts" lead to completely different designs.

2.2 Option 2: medium ISR

The handler directly updates the delay and flash-mode variables. The tasks read them and drive the LEDs.

Quick, low overhead, no protocol needed. There is no accumulation problem because the shared variables are "latest value wins" by nature: if the user mashes the switch five times before a task runs, the ISR writes five times and the task reads the fifth, which is exactly right.

2.3 Option 3: long ISR

The handler updates the variables *and* immediately drives the LEDs.

Most responsive, and it breaks. When the handler returns, the preempted task resumes exactly where it left off, mid-sequence, and lights the LEDs with the old colours or the old delays, undoing the handler's work. To make this correct you need a way to abort or restart the interrupted task, and that is not simple without a kernel.

The verdict

The book chooses **option 2**, the medium ISR, and states the general principle: more partitioning improves responsiveness but increases complexity. The designer's job is to find the balance.

Option 3's failure is the interesting one, because it is not about speed. It is that an ISR which touches state a task is mid-way through using creates an *inconsistency*, and inconsistency is worse than latency.

3 Deep dive 2: Writing the handler

Now the code. The rules are short:

- **Takes no arguments, returns nothing**: `void EXTI0_IRQHandler(void)`.
- **Must be named exactly** as the startup file's vector table expects, so the linker puts its address in the right slot.
- Recall from Week 07 that the hardware already stacked the caller-saved registers, so **no special keyword or attribute is needed**. It is an ordinary C function that nothing calls.

Listing 1: Book Listing 4.9. One handler, two switches, twelve possible sources.

```
volatile uint8_t  g_flash_LED = 0;
volatile uint32_t g_w_delay   = W_DELAY_SLOW;
volatile uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void EXTI4_15_IRQHandler(void) {
    if ((EXTI->PR & MASK(SW1_POS)) != 0) {
        EXTI->PR = MASK(SW1_POS);           /* clear pending, write-1 */
        if (SWITCH_PRESSED(SW1_POS)) {
            g_flash_LED = 1;                /* flash white  */
        } else {
            g_flash_LED = 0;                /* RGB sequence */
        }
    }
    if ((EXTI->PR & MASK(SW2_POS)) != 0) {
        EXTI->PR = MASK(SW2_POS);
        if (SWITCH_PRESSED(SW2_POS)) {
            g_w_delay = W_DELAY_FAST;  g_RGB_delay = RGB_DELAY_FAST;
        } else {
            g_w_delay = W_DELAY_SLOW;  g_RGB_delay = RGB_DELAY_SLOW;
        }
    }
    EXTI->PR = 0x0000FFF0;    /* clear all other lines 4..15 */
}
```

Walk the structure. Because the F091 routes twelve EXTI lines to this one handler, the handler cannot know why it was called. So it **reads `EXTI_PR` and tests each bit it cares about**. That is the demultiplexing step, and any grouped handler needs it.

For each line that did request: clear the pending bit by writing a 1, then read the current pin level from `IDR` via the `SWITCH_PRESSED` macro, then update the shared variables.

Beware the trap

Notice the last line, `EXTI->PR = 0x0000FFF0`, which clears the pending flags for every line from 4 to 15 whether or not it was serviced. `0xFFF0` is binary `1111 1111 1111 0000`.

The book's own exercise 14 asks when this might cause incorrect operation, and the answer is worth having: **if another peripheral or pin on lines 4–15 requested an interrupt that this handler does not service, that request is silently discarded.** The event is lost, not deferred.

It is safe here only because these two switches are the only users of that range. In a system where they are not, this line is a bug that deletes other people's interrupts. Do not copy it reflexively.

Book board vs your board

Recall from Week 07 that your F303 gives lines 0–4 their own handlers, so `EXTI4_15_IRQHandler` does not exist on your part. Your button on PA0 uses `EXTI0_IRQHandler`.

The upside is that a dedicated handler **needs no demultiplexing**. It already knows which line fired, so the `if ((EXTI->PR & ...))` test collapses away. Your lab manual's pattern is the HAL version of exactly this:

```c
void EXTI0_IRQHandler(void) {
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);    /* clears the flag for you
        */
}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == GPIO_PIN_0) {
        /* your work here */
    }
}
```

The `if` inside the callback is still necessary, though, because `HAL_GPIO_EXTI_Callback` is shared by *every* EXTI line in the whole program. Its `GPIO_Pin` argument is HAL's demultiplexing, moved one level up.

For your grouped handlers, `EXTI9_5_IRQHandler` and `EXTI15_10_IRQHandler`, you must pass every pin you care about:

```c
void EXTI9_5_IRQHandler(void) {
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_5 | GPIO_PIN_7);
}
```

4 Deep dive 3: Corruption type 1, the stale register

Right. Now the interesting part.

Here is the setup. A shared variable lives in memory. To do anything with it, the CPU must load it into a register first, because Cortex-M arithmetic works on registers, not memory. Recall that from Week 06.

Now, if the variable appears three times in a function, does the compiler emit three loads?

Not if it can help it. A compiler's job includes eliminating redundant work, and from its point of view a second load of a variable nothing has written to is pure waste. So it loads once and reuses the register.

Which is correct, unless something outside the compiler's view changes memory.

Listing 2: Book Listing 4.10. Three uses, possibly one load.

```c
void Task_RGB(void) {
    if (g_flash_LED == 0) {
        Control_RGB_LEDs(1, 0, 0);
        Delay(g_RGB_delay);    /* loads g_RGB_delay into a register */

        /* <-- If switch 2 changes NOW, the ISR runs and updates
               g_RGB_delay in MEMORY. The register still holds the old value.
                    */

        Control_RGB_LEDs(0, 1, 0);
        Delay(g_RGB_delay);    /* ERROR: may reuse the stale register */
        Control_RGB_LEDs(0, 0, 1);
        Delay(g_RGB_delay);    /* ERROR: same */
    }
```

```
}
```

The ISR updated memory. The task is reading a register. They have diverged, and the task carries on with the old timing until the function returns.

There is a mirror-image version on the write side, and it is worse. The compiler may also defer *storing* a value back to memory until the end of a function, again to avoid redundant work. So: your function loads the variable, modifies its register copy, and has not yet written back. An interrupt fires. The ISR writes a new value to memory. Your function resumes, finishes, and stores its register, **overwriting the ISR's new value with a modified version of the old one**. The ISR's work is silently erased.

`volatile`

> The keyword that tells the compiler a variable may change outside the program's normal flow of control. It forces a reload from memory at every use in the source, and a store at every assignment, defeating exactly the optimisations above.
>
> Data is volatile if **an ISR may change a variable used by main-line code**, or if it is a hardware register that changes on its own, like a counter or a status flag.

Listing 3: Book Listing 4.11. The fix, and it is only three keywords.

```
volatile uint8_t  g_flash_LED = 0;
volatile uint32_t g_w_delay   = W_DELAY_SLOW;
volatile uint32_t g_RGB_delay = RGB_DELAY_SLOW;
```

Recall from Week 03 that CMSIS marks every peripheral register `__IO`, which expands to `volatile`, for the second reason: a status register changes when the hardware feels like it, not when your code writes to it.

Beware the trap

> The symptom of a missing `volatile` is uniquely maddening: **it works in a debug build and fails in a release build.** At `-O0` the compiler does not bother caching values, so the bug is invisible. Turn on `-O2` and it appears.
>
> So if your program works when you step through it and misbehaves when you let it run, or works in Debug and fails in Release, check every variable shared between an ISR and main-line code for `volatile` before you look anywhere else.

5 Deep dive 4: Corruption type 2, the read-modify-write race

Now the one that matters most, because `volatile` does not fix it and almost everyone assumes it does.

Take the simplest possible shared variable and the simplest possible operation.

Listing 4: Book Listing 4.12. Both increment. It is `volatile`. It is still broken.

```
volatile uint32_t g_counter;

void my_ISR(void)           { g_counter++; }
void my_bad_function(void)  { g_counter++; }
```

Trace it instruction by instruction. Recall from Week 03 that `x++` on a memory variable is not one operation, it is three: load, add, store.

	`my_bad_function`	`my_ISR`	`g_counter` **in memory**
1	`LDR R0, [g_counter]` $\Rightarrow R0 = 0$		0
2	**interrupt fires here**		0
3		`LDR` $\Rightarrow R0 = 0$	0
4		`ADDS` $\Rightarrow R0 = 1$	0
5		`STR`	**1**
6	`ADDS R0, #1` $\Rightarrow R0 = 1$	(returned)	1
7	`STR R0, [g_counter]`		**1**

Two increments happened. The counter went from 0 to 1. One increment vanished.

Data race and critical section

Data race: a situation in which the ill-timed preemption of a critical section produces an incorrect result. Whether it happens depends on the precise timing relationship between the program and the interrupt, which is why these bugs are intermittent.

Critical section: a section of code that may execute incorrectly if not executed **atomically**, meaning all-or-nothing with no possibility of interruption partway through.

Beware the trap

This is the single most misunderstood point in the chapter, so state it precisely.

`volatile` guarantees that each individual read and each individual write actually goes to memory. It guarantees **nothing whatsoever** about the atomicity of a sequence of them.

`g_counter` in the table above *is* `volatile`. Every load and store really did hit memory. The race happened anyway, because the race is between three separate memory operations, and `volatile` has no opinion about what happens between them.

`volatile` **fixes staleness. It does not fix races.** If an exam asks whether `volatile` makes a shared counter safe, the answer is no, and this table is the reason.

5.1 The fix

If the problem is that the sequence can be interrupted, the fix is to make it uninterruptible. Recall the save-modify-restore discipline from Week 07:

Listing 5: Book Listing 4.12, the good version.

```
void my_good_function(void) {
    uint32_t masking_state;

    masking_state = __get_PRIMASK();    /* remember previous state */
    __disable_irq();

    g_counter++;                        /* critical section */

    __set_PRIMASK(masking_state);       /* restore, do not assume */
}
```

Three instructions of protected work, four instructions of protection. That ratio is normal and it is the price.

Why not just use `BSRR`-style atomic hardware?

Sometimes you can, and you should. Recall from Week 03 that `GPIOx->BSRR` exists precisely so a pin change needs no read-modify-write, and from Week 07 that `EXTI->PR` is write-1-to-clear for the same reason.

When the hardware offers an atomic operation, use it and skip the critical section entirely. Critical sections are for arithmetic on ordinary variables, where no such hardware exists.

In the lab

Your self-balancing robot is full of these, and one is worth calling out specifically.
Suppose your timer ISR increments a millisecond tick counter and your control loop reads it to compute Δt. That is one writer and one reader of a 32-bit variable, which happens to be safe on this CPU: a single aligned 32-bit load or store is atomic on Cortex-M, so a reader either sees the old value or the new one, never a mixture.
But suppose your ISR instead accumulates gyro samples into a running sum and a counter, and the control loop reads both and resets them. Now there are **two** variables that must be consistent with each other, and the read-and-reset is itself a read-modify-write. If the ISR fires between reading the sum and resetting it, you lose a sample, and your angle estimate drifts by a small amount, every time, in one direction. That is exactly the kind of drift people misdiagnose as gyro bias for an entire evening.

6 Deep dive 5: Corruption type 3, the partial update

The third failure needs no arithmetic at all. A plain read of a plain variable can return garbage that was never written.

The condition is that the variable is **larger than one word**. On Cortex-M a word is 32 bits, so a `uint64_t` takes two loads to read and two stores to write.

Say you widen `g_RGB_delay` from `uint32_t` to `uint64_t` to allow longer delays. Reading it now compiles to:

```
1 LDR R0, [g_RGB_delay]        /* low word  */
2 LDR R1, [g_RGB_delay + 4]    /* high word */
```

If switch 2 changes between those two instructions, the ISR runs and updates *both* words in memory. Your task then loads the high word. Result: **R0 holds the old low word and R1 holds the new high word**, a value that is neither the old delay nor the new one and may be absurd.

The rule that falls out of this

On a 32-bit Cortex-M, a naturally aligned **32-bit or smaller** load or store is a single instruction and therefore atomic. A reader sees either the whole old value or the whole new one.

Anything **larger than 32 bits**, and any **struct or array** where several fields must agree, needs a critical section around *both* the write and the read.

Practical consequence: **prefer 32-bit or smaller types for variables shared with an ISR.** A `uint64_t` tick counter or a `double` shared with a handler is a partial-update waiting to happen. This also means `float` is fine on size grounds, 32 bits, while `double` is not.

Beware the trap

The partial-update problem is not limited to wide scalars. Consider a struct holding an x, y, z accelerometer reading, written by an ISR and read by your control loop. Each field is

16 bits and individually atomic, so nothing is ever half-written.

And the reader can still get x and y from sample n and z from sample $n{+}1$, which is a reading of an orientation the robot was never in. Every individual field is fine. The *set* is inconsistent.

This is the same disease as the 64-bit case with a different shape, and the same fix: protect the whole group, or use a scheme where the reader can detect that it was interrupted and retry.

7 Deep dive 6: The three failures, side by side

Since the whole point is telling them apart, here they are together.

Failure	Cause	Fixed by	Symptom
Stale register	Compiler caches a value in a register, or defers a store	`volatile`	Works at `-O0`, fails at `-O2`. Debug fine, Release broken
Read-modify-write race	Load, modify, store is three operations and can be split	Critical section, or atomic hardware	Intermittent. Counts drift slowly. Not reproducible
Partial update	Value $>$ 32 bits, or several fields that must agree	Critical section, or use $\leq$ 32-bit types	Occasional absurd value from nowhere

The two-part answer to give

When asked how to share data safely between an ISR and main-line code:

1. Declare it `volatile`, so the compiler actually reads and writes memory.
2. Protect every multi-step access with a critical section, or use hardware that is atomic by construction.

Both. Neither alone is sufficient, and naming only `volatile` is the standard wrong answer.

Note also that interrupts are not the only source of preemption. In a system with a **preemptive kernel**, tasks preempt each other too, so all three failures apply between two tasks with no interrupt involved. Recall the preemption discussion from Week 05. A real kernel provides mutexes and semaphores as better-behaved alternatives to switching off all interrupts, precisely because `__disable_irq` is so blunt.

7.1 The cost of a critical section

One number to keep in mind. Recall from Week 07 that `__disable_irq` stops *every* maskable interrupt, and that critical-section length adds directly to the worst-case latency of every interrupt in the system.

So the two design pressures point in opposite directions. Correctness says protect the shared data. Responsiveness says never block interrupts. The resolution is not to pick a side, it is to make critical sections **as short as physically possible**: copy the shared value into a local, leave the critical section, then do the work on the local.

Listing 6: The pattern. Protect the access, not the processing.

```
/* Bad: the whole computation runs with interrupts off */
__disable_irq();
result = expensive_filter(g_samples, g_count);   /* microseconds! */
__set_PRIMASK(s);

/* Good: snapshot under protection, compute outside it */
s = __get_PRIMASK();
__disable_irq();
local_count = g_count;                      /* a few cycles */
memcpy(local_samples, g_samples, sizeof local_samples);
g_count = 0;
__set_PRIMASK(s);

result = expensive_filter(local_samples, local_count);  /* unprotected */
```

And that is a wrap for Chapter 4. Take a breath, the midterm is next.

8 Tying it back

We asked what happens when a handler lands in the middle of your task's arithmetic, and found three distinct answers.

The compiler may have cached the variable in a register, so your task carries on with a value the ISR already replaced in memory, or writes back a stale one and erases the ISR's work. That is staleness, and `volatile` is the whole fix, because `volatile` forces a real memory access at every use.

Or the operation itself was three instructions, load, modify, store, and the handler slipped between them, so two increments produced one. That is a data race, and `volatile` does not touch it, because every one of those three accesses did go to memory exactly as promised. The fix is a critical section, saving and restoring `PRIMASK` rather than assuming it, or better, using hardware that is atomic by design, which is what `BSRR` and write-1-to-clear pending flags were for.

Or the variable was wider than a word, or was several fields that had to agree, and the reader got half of one value and half of another. That is a partial update, fixed by a critical section or by not sharing anything bigger than 32 bits.

Wrapped around all three is the partitioning decision: keep the handler short so it does urgent work only, defer the rest, and choose your handoff mechanism according to whether your data is "latest value wins" or "every value counts", because the second kind needs a queue.

Which is where Chapter 4 ends and the midterm begins. Weeks 1 through 4 of the syllabus are Chapters 1 to 4: logic levels, GPIO, architecture, and the NVIC. After that, Chapter 5 opens the compiler and shows you what your C has actually been turning into all along, which is a good place to be standing when we start caring about cycle counts.

Cheat sheet: Week 08

Partitioning an interrupt's work

Rule: the ISR does **quick, urgent** work only; defer the rest to main-line code.
Short ISR: just record the event. Needs a handoff protocol, risks wrecking modularity.
Medium ISR: update the shared variables. *Usually the right answer.* Low overhead, no protocol.
Long ISR: also drive the outputs. Most responsive, but the preempted task resumes and undoes it.

Latest value wins (switch position) ⇒ plain shared variables are fine.
Every value counts (serial bytes) ⇒ you need a **queue**. Week 14.

Writing the handler

`void NAME_IRQHandler(void)` — no arguments, no return, **name must match the startup file**.
No special keyword needed: the hardware already stacked the caller-saved registers.
A **grouped** handler must demultiplex by reading the pending register and testing each bit.
Clear only the flags you serviced. `EXTI->PR = 0x0000FFF0` discards other lines' requests entirely — safe only if nothing else uses that range.
HAL: `HAL_GPIO_EXTI_IRQHandler(pin)` clears the flag, then `HAL_GPIO_EXTI_Callback` runs — and that callback is shared by *every* line, so keep the `if (GPIO_Pin == ...)`.

The three corruptions, and this table is the exam answer

1. Stale register. Compiler caches the value, or defers the store. → `volatile`.
Symptom: works at `-O0` / Debug, fails at `-O2` / Release.

2. Read-modify-write race. `x++` is load, add, store. Interrupt lands between them, one increment vanishes. → **critical section** or atomic hardware.
Symptom: intermittent, drifting counts, not reproducible.

3. Partial update. Value $>$ 32 bits, or several fields that must agree. Reader gets half old, half new. → **critical section**, or keep shared types $\leq$ 32 bits.
Symptom: occasional impossible value.

The point people get wrong

`volatile` guarantees each read and each write reaches memory. It guarantees **nothing** about atomicity across a sequence of them.
`volatile` **fixes staleness. It does not fix races.**
A `volatile` counter incremented by both an ISR and main code **is still broken**.

Safe answer to "how do I share data with an ISR": **(1)** `volatile`**, and (2) a critical section around every multi-step access.** Both.

Atomicity on Cortex-M

A naturally aligned load or store of **32 bits or fewer is atomic** — one instruction.
So a single-writer, single-reader `uint32_t` or `float` needs no critical section, only `volatile`.
`uint64_t`, `double`, structs, and arrays are **not** atomic. Prefer $\leq$ 32-bit shared types.
Where hardware provides atomicity, use it instead: `GPIOx->BSRR`, `EXTI->PR` write-1-to-clear, `NVIC->ISER`.

Critical sections, correctly and cheaply

`s = __get_PRIMASK(); __disable_irq(); /* ... */ __set_PRIMASK(s);`

Save, modify, restore. Never `__enable_irq()` unconditionally — it breaks a caller's critical section.

Length adds directly to **worst-case latency of every interrupt**. So: **snapshot inside, compute outside.** Copy shared data to locals under protection, then process the locals unprotected.

Preemptive kernels have the same three problems between tasks, and offer mutexes and semaphores as less blunt tools than disabling all interrupts.

Likely exam prompts from this section

· Does `volatile` make a shared counter safe? **No.** Explain with the load/add/store trace. *Near certain.*
· Give the three ways shared data gets corrupted and the fix for each.
· Write a correct critical section and say why the naive version is wrong.
· Why does a bug appear only in an optimised build?
· Under what circumstances does clearing all pending flags in `EXTI_PR` cause incorrect operation?
· Compare short, medium, and long ISR partitioning, and say why the long version fails.